Inductively Defining Programming Languages

The following BNF equations describe a simple programming language.

$$
\begin{array}{rcl}
p & ::= & x := e \\
& | & p_1 ; p_2 \\
& | & \texttt{if } b \texttt{ then } p_1 \texttt{ else } p_2 \\
& | & \texttt{while } b \texttt{ do } p
\end{array}
\qquad
\begin{array}{rcl}
b & ::= & \texttt{true} \\
& | & \texttt{false} \\
& | & b_1 \texttt{ or } b_2 \\
& | & e_1 = e_2 \\
& | & \cdots
\end{array}
\qquad
\begin{array}{rcl}
e & ::= & n \\
& | & e_1 + e_2 \\
& | & e_1 - e_2 \\
& | & \cdots
\end{array}
$$

(The above list-like layout is how BNF equations are typically written in programming language definitions.)

There are three syntax classes being defined: programs p, Boolean expressions b; and numerical expressions e. Each of these classes is defined inductively. x ranges over numerical program variables.

A Simple Programming Languages

A program in the programming language given by the above BNF equation is either

- an **assignment** statement "$x := e$" which evaluates the integer expression e and assigns this value to the variable x; or
- the **sequential composition** "$p_1 ; p_2$" of two (smaller) programs p_1 and p_2 which first executes the program p_1, and then executes the program p_2 if and when program p_1 has terminated; or
- a **conditional** statement "`if` b `then` p_1 `else` p_2" involving a Boolean test b and two (smaller) programs p_1 and p_2 which first evaluates the Boolean expression b, and then either executes the program p_1 if b evaluated to true, or executes the program p_2 if b evaluated to false; or
- a **while loop** "`while` b `do` p" involving a Boolean test b and a (smaller) program p which repeatedly executes the program p for as long as the Boolean test b is true; that is, it first evaluates the Boolean expression b, and then either terminates if b evaluated to false, or executes the program p and repeats itself (starting with re-evaluating the Boolean expression b) if b evaluated to true.

An Example Program

The following is a program in this language for computing the sum of the first n positive integers: $s = 1 + 2 + 3 + \cdots + n$.

```
i := 0;
s := 0;
while i<n do
  { i := i+1;
    s := s+i }
```

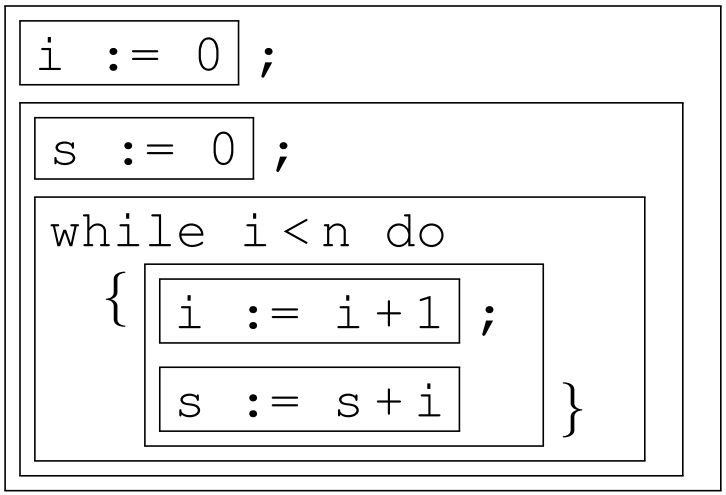
This 5-line program consists of two smaller programs combined with the sequential composition symbol:

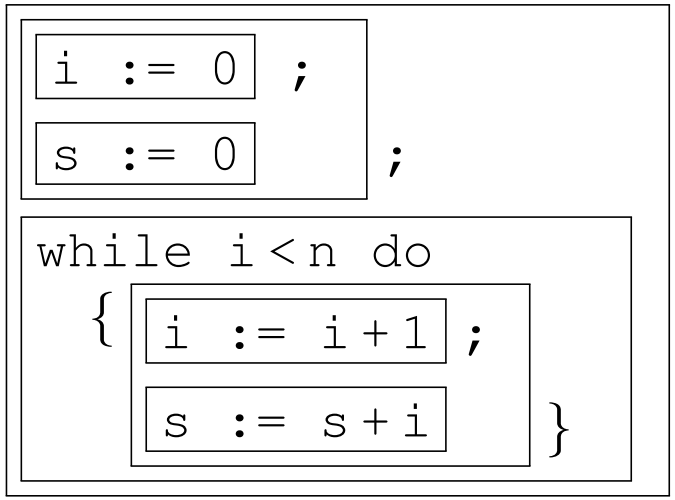
```
i := 0 ;
s := 0;
while i<n do
  { i := i+1;
    s := s+i }
```

Note that we also include braces to help us parse the program.

Decomposing Our Program

The whole program can be decomposed into its basis elements. In fact, it is possible to parse this program in different ways:





In the first case, we have an assignment statement sequentially combined with the sequential composition of two smaller programs.

In the second case, we have two programs combined with the sequential composition symbol, the first being itself two simple assignment statements composed together sequentially, and the second being a while loop.

Inductively-Defined Data Types

Data types in programming languages are typically defined inductively.

Example: A list of integers is defined by the BNF equation:

$$\mathrm{L} \quad ::= \quad [\,] \quad | \quad \mathrm{n} : \mathrm{L}$$

In this definition, n ranges over integers, and the colon symbol ":" represents the operation of adding an element to the front of a list.

Thus, a list is either

1. the empty list $[\,]$ (ie, the list containing no items); or
2. a list obtained by adding a number n to the head of a (smaller) list L.

The first element of a non-empty list is called its *head*, and the rest of the list is called its *tail*.

For example, $[1,2,3] \;=\; 1 : (\,2 : (\,3 : [\,]\,)\,)$.

- The head of this list is 1;
- The tail of this list is $[2,3]$.

Inductively-Defined Binary Trees

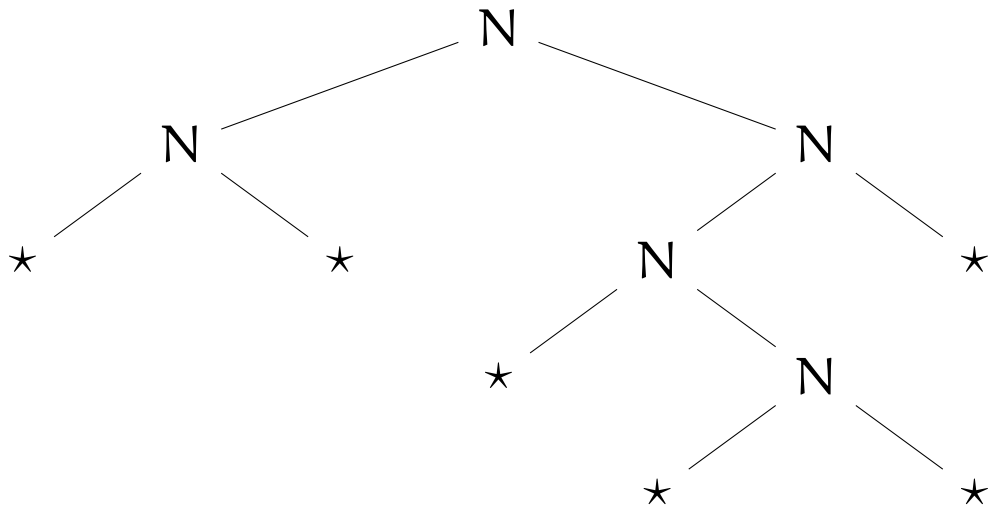
A binary tree is defined by the BNF equation:

$$t \ ::= \ \star \ \mid \ N(t_1, t_2)$$

That is, a tree is either a leaf $\star$ or an internal node $N(t_1, t_2)$ with two subtrees t_1 and t_2. For example, the tree

$$N(N(\star, \star), N(N(\star, N(\star, \star)), \star))$$

may be represented by the following picture:



Inductively-Defined Sequences

A sequence $a_0, a_1, a_2, a_3, a_4, \ldots$ is a function whose domain is the (inductively-defined) natural numbers.

The ***factorial function*** $n! = 1 \times 2 \times 3 \times \cdots \times n$ is defined inductively as follows

$$0! = 1; \quad \text{and} \quad n! = n \times (n-1)! \quad \text{for } n > 0.$$

Thus, for example,

$$3! \;=\; 3 \times 2! \;=\; 3 \times (2 \times 1!) \;=\; 3 \times 2 \times (1 \times 0!) \;=\; 3 \times 2 \times 1 \times 1 = 6$$

The ***Fibonacci numbers*** are defined inductively by:

$$f_0 = 0;\ f_1 = 1; \quad \text{and} \quad f_n = f_{n-1} + f_{n-2} \quad \text{for } n > 1.$$

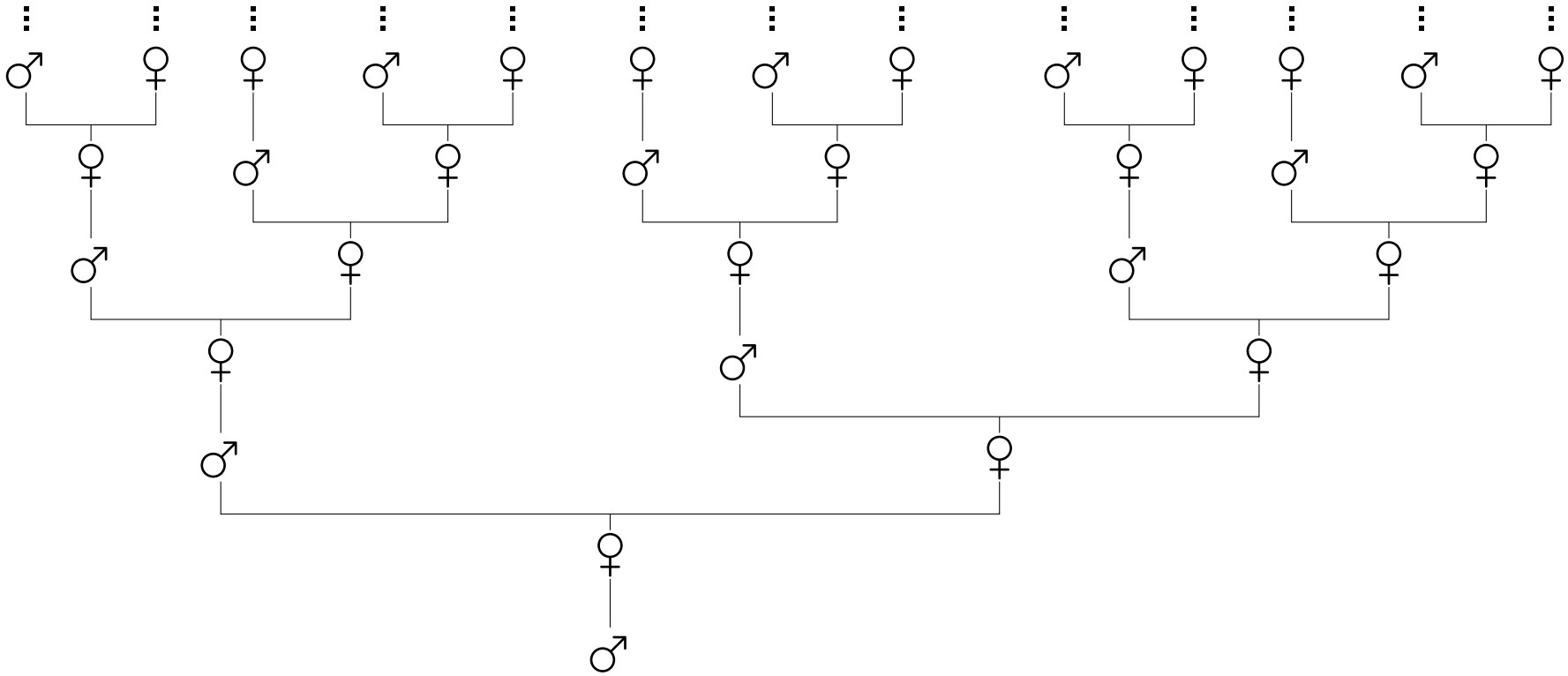
The first few Fibonacci numbers are $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \ldots$

The ***Harmonic numbers*** $H_n = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$ are defined inductively by:

$$H_0 = 0; \quad \text{and} \quad H_n = H_{n-1} + \frac{1}{n} \quad \text{for } n > 0.$$

Example: Inductively-Bred Bees

Male bees hatch from unfertilised eggs, and so have a mother but no father.
Female bees hatch from fertilised eggs, and so have both a mother and a father.



How many ancestors does a male bee have in the tenth generation back?
How many of these ancestors are male?

Inductively-Defined Functions

Functions of inductively-defined sets are naturally defined inductively. That is, to define a function on an inductively-defined data type:

- first specify its value for the basic data;
- then define its value on other data in terms of its value on smaller data.

The ***length of a word*** $w \in A^*$ is defined inductively as

$$\mathit{length}\ (\varepsilon) = 0 \qquad \text{and} \qquad \mathit{length}\ (aw) = 1 + \mathit{length}\ (w)$$

The ***length of a list*** is defined inductively as

$$\mathit{length}\ ([\,]) = 0 \qquad \text{and} \qquad \mathit{length}\ (a : L) = 1 + \mathit{length}\ (L)$$

The ***height of a binary tree*** is defined inductively as

$$\mathit{height}\ (\star) = 0 \quad \text{and}$$
$$\mathit{height}\ (N(t_1, t_2)) = 1 + \max\left(\ \mathit{height}\ (t_1)\ , \mathit{height}\ (t_2)\ \right)$$

Example Inductive Functions

We can add two natural numbers inductively as follows:

$$\mathrm{add}\,(m, 0) = m \qquad \text{and} \qquad \mathrm{add}\,(m, n{+}1) = \mathrm{add}\,(m, n) + 1$$

Thus, for example,

$$\mathrm{add}\,(3,2) \;=\; \mathrm{add}\,(3,1) \;+\; 1 \;=\; \mathrm{add}\,(3,0) \;+\; 1 \;+\; 1 \;=\; 3 \;+\; 1 \;+\; 1 \;=\; 5$$

We can inductively find the maximum of a non-empty list of numbers as follows:

$$\mathrm{listmax}\,(\,[\,n\,]\,) = n; \quad \textit{and}$$
$$\mathrm{listmax}\,(\,k : (n : L)\,) = \max\,(\,k\,,\ \mathrm{listmax}\,(n : L)\,).$$

Thus, for example,

$$\begin{aligned}
\mathrm{listmax}\,(\,[\,2\,,\,6\,,\,5\,]\,) &= \max\,(\,2\,,\ \mathrm{listmax}\,(\,[\,6\,,\,5\,]\,)\,) \\
&= \max\,(\,2\,,\ \max\,(\,6\,,\ \mathrm{listmax}\,(\,[\,5\,]\,)\,)\,) \\
&= \max\,(\,2\,,\ \max\,(\,6\,,\,5)\,) \\
&= \max\,(\,2\,,\,6\,) \\
&= 6.
\end{aligned}$$

Appending Lists Inductively

The append function $L_1 ++ L_2$ is defined inductively as

$$[\,] ++ L = L \qquad \text{and} \qquad (a : L_1) ++ L = a : (L_1 ++ L)$$

That is,

- appending the empty list $[\,]$ to a list L doesn't change the list L;
- to append the list $(a : L_1)$ to the list L, we first append L_1 to L, and then we add a to the front of the result.

Example:
$$\begin{aligned}
[1,2] ++ [3,4,5] &= (1 : (2 : [\,])) ++ [3,4,5] \\
&= 1 : ((2 : [\,]) ++ [3,4,5]) \\
&= 1 : (2 : ([\,] ++ [3,4,5])) \\
&= 1 : (2 : [3,4,5]) \\
&= [1,2,3,4,5]
\end{aligned}$$

Recursive Procedures

Programs process inductively-defined data recursively.

For example, in order to sort a non-empty list (n:L), you can sort the tail of the list L, and then insert the head of the list n into the correct place.

This procedure – called *insertion sort* – is based on the following inductive definition:

$$\begin{aligned} \mathit{isort}\ [\,] &= [\,] \\ \mathit{isort}\ (\mathrm{a} : \mathrm{L}) &= (\mathit{insert}\ \mathrm{a})\ (\mathit{isort}\ \ \mathrm{L}) \end{aligned}$$

The first (base) clause says that the empty list is already sorted; while the second (inductive) clause uses the following inductive definition for inserting an element into a sorted list:

$$\begin{aligned} (\mathit{insert}\ \mathrm{a})\ [\,] &= [\mathrm{a}] \\ (\mathit{insert}\ \mathrm{a})\ (\mathrm{b} : \mathrm{L}) &= \textit{if}\ \mathrm{a}{<}\mathrm{b}\ \textit{then}\ \mathrm{a} : (\mathrm{b} : \mathrm{L}) \\ &\quad\ \ \textit{else}\ \mathrm{b} : ((\mathit{insert}\ \mathrm{a})\ \mathrm{L}) \end{aligned}$$